

SeuGado — João: o plano semanal

Otimizador guloso semanal: do estado do pasto à recomendação de manejo

27/09/2026

Sua parte em uma frase: o cérebro da recomendação. Você recebe a fotografia da fazenda (quanto capim cada piquete tem hoje, quanto vai crescer em cada um dos próximos dias e quanto cada lote come) e devolve o **plano da semana**: quais lotes mudar de piquete, em que dia, para onde, por quê e com que confiança.

Você entrega: `planner/otimizador.py`, `planner/confianca.py`, `planner/geo.py` e `persistencia/planos.py`, com testes. Seu código não tem tela nem rede; é Python puro, mais duas funções de banco.

Prazo: PR aberto até domingo à noite.

1. O SeuGado em um minuto

O gado é dividido em **lotes**, e o pasto em **piquetes**. O capim tem um ponto certo para o lote **entrar** (altura de entrada) e um ponto em que o lote precisa **sair** (altura de saída). Se o gado entra tarde, o capim já passou do ponto e perdeu valor. Se sai tarde, a planta fica rapada demais e o pasto se degrada por temporadas. **Esse é o erro mais caro.**

O Kauê estima o capim de cada piquete por satélite e clima. O Leo manda o plano pelo Telegram. **Você decide o plano.** Na call, a equipe combinou:

- o plano é **semanal** (7 dias a partir de hoje);
- movimentação **só nos dias de manejo preferidos** pelo produtor (ex.: segunda e quinta);
- no máximo `funcionarios × manejos_por_funcionario_dia` movimentações por dia.

Isto é o **estágio 1** do motor de otimização do projeto (guloso, ADR-008). Busca local e CP-SAT vêm depois; o seu guloso vira a linha de base contra a qual eles serão medidos.

Palavras que você vai usar

Termo	O que é
Massa (kg MS/ha)	Quanto capim seco há por hectare. É o “estoque” do piquete
Altura (cm)	$\text{massa} \div \text{densidade}$. O produtor pensa em cm; toda comparação com alvo é em cm
Taxa de acúmulo (kg MS/ha/dia)	Quanto o capim cresce por dia. Vem pronta, uma por dia, para 14 dias
Consumo do lote (kg MS/dia)	Quanto o lote come por dia. Vem pronto
Eficiência de pastejo	Fração do capim removido que vira comida; o resto é pisoteado. Remoção do piquete por dia = $\text{consumo} \div \text{eficiência} \div \text{área}$

Termo	O que é
Descanso	Dias desde que o último lote saiu. O piquete só recebe lote depois de um descanso mínimo (21 dias no Marandu)
Apto	Piquete que pode receber lote: altura $\geq$ entrada e descanso cumprido e vazio
Urgência	altura_saida - altura_prevista. Quanto maior, mais urgente tirar o lote
Confiança	alta / media / baixa. A recomendação vale o que vale o dado mais fraco (ADR-019)
Rotacionado / contínuo	Só piquetes rotacionados entram no plano. No contínuo, o lote fica fixo, e você só gera alerta

2. Stack e onde fica o seu código

Python 3.12, sem dependência nova. Use o que já existe:

Onde	O que usar
seugado.contratos	Entrada: EstadoProjetado, PiqueteProjetado, LoteProjetado. Saída: PlanoManejo, Movimentacao, Alerta, PedidoValidacao, ResumoPiquete, TipoAlerta. Mais plano_para_dict/plano_de_dict e estado_de_dict
seugado.core.models	Fazenda (configuração), Confianca, MetodoPastejo
seugado.core.regras	urgencia, descanso_cumprido, combinar_confianca
seugado.core.forragem	massa_para_altura, dias_ocupacao
seugado.planner.estado	avancar_massa_um_dia (a física de um dia, a mesma que o Kauê usa)
seugado.persistencia.eventos	registrar_evento

Arquivos que são seus: src/seugado/planner/otimizador.py, src/seugado/planner/confianca.py, src/seugado/planner/geo.py, src/seugado/persistencia/planos.py, tests/planner/test_otimizador.py, tests/planner/test_confianca.py, tests/persistencia/test_planos.py.

Não edite nenhum outro arquivo. Se precisar de algo em contratos.py ou core/, fale com o Kauê.

Regras de código do projeto (valem para você também)

- Type hints em toda função pública; nomes com unidade (altura_cm, massa_kg_ms_ha, area_ha).
- Termos do domínio em português (piquete, lote); docstrings e comentários em inglês.
- Nunca use entidade como chave de dicionário, só o id. Nunca compare membro de um enum com outro enum.
- Arquivo com mais de ~300 linhas → quebre em funções auxiliares em outro arquivo seu.
- Sem aleatoriedade: **mesma entrada → mesmo plano.**

3. Preparar o ambiente

Importante: o Python não lê o `.env` sozinho. Todo comando `uv run local` leva `--env-file .env` (como nos exemplos abaixo); sem isso a API responde erro 500 e os testes de banco são pulados.

1. Instale Git e **uv** (docs.astral.sh/uv). Clone o repositório e crie a sua branch: `feat/joao-otimizador`.
2. `uv sync --group dev`.
3. Rode `uv run --env-file .env pytest` para confirmar que tudo passa antes de você mexer.
4. Para `persistencia/planos.py` (tarefa J5), crie o `.env` a partir de `.env.example` com o `DATABASE_URL` e o `SEUGADO_TEST_DATABASE_URL` que o Kauê manda em privado.

Antes do PR: `uv run ruff check .`, `uv run mypy` e `uv run --env-file .env pytest` sem erro.

4. O contrato: o que entra e o que sai

```
def gerar_plano(
    estado: EstadoProjetado,
    fazenda: Fazenda,
    agora: datetime,           # timezone-aware (UTC); vira plano.data_geracao
    plano_id: UUID,           # quem chama gera (uuid4); nos testes, fixo
) -> PlanoManejo: ...
```

Entrada (`tests/fixtures/estado_projetado_exemplo.json` tem um exemplo completo; carregue com `estado_de_dict(json.load(...))`):

- `estado.data_base`: o primeiro dia do plano (hoje).
- Para cada piquete: `area_ha`, `metodo_pastejo`, `situacao`, `lote_atual_id`, `dias_descanso`, `parametros` (alturas de entrada e saída, ou máxima e mínima no contínuo), `faltantes`, `densidade_kg_ha_por_cm`, `eficiencia_pastejo`, `massa_hoje_kg_ms_ha`, `altura_hoje_cm`, `taxa_acumulo_prevista_kg_ms_ha_dia` (**14 valores**: índice 0 = hoje), `centroide_lat/lon`, `confianca`, `motivo_confianca`.
- Para cada lote: `piquete_atual_id`, `consumo_kg_ms_dia`, `indissoluvél`, `confianca_peso`, `motivo_confianca_peso`.
- `fazenda.dias_preferenciais_manejo`: `date.weekday()` (0 = segunda ... 6 = domingo); `funcionarios_disponiveis`; `manejos_por_funcionario_dia`.

Saída: `PlanoManejo` (exemplo completo em `tests/fixtures/plano_exemplo.json`). **Esse arquivo é exatamente o que o seu `gerar_plano` deve produzir para o estado de exemplo**, com a fazenda do exemplo (dias preferenciais segunda e quinta, 1 funcionário, 2 manejos por dia), `agora = 2026-09-28T08:00:00+00:00` e `plano_id = 44444444-4444-4444-8444-000000000001`. Ele é o seu teste de aceitação principal.

5. Tarefas (na ordem)

J1: Distância (planner/geo.py)

Por quê: entre dois piquetes igualmente bons, o gado vai para o mais perto, o que evita rotas cruzadas pela fazenda (ADR-016). No MVP, a distância é em linha reta entre os centros dos piquetes.

Como: `def distancia_m(lat1: float, lon1: float, lat2: float, lon2: float) -> float`, pela fórmula de **Haversine** com raio da Terra de 6.371.000 m.

Critérios de aceite:

- [] Mesmo ponto → 0.
- [] 0,001° de latitude ≈ 111,2 m (± 0,5).

Fora do escopo: rota por porteira ou corredor; PostGIS.

J2: Confiança da movimentação (planner/confianca.py)

Por quê: toda recomendação diz o quanto dá para confiar nela e **por quê**, nomeando o elo mais fraco (ADR-019). É o que responde ao produtor quando ele pergunta “como eu sei que isso tá certo?”.

Como:

```
def confianca_movimentacao(destino: PiqueteProjetado, lote: LoteProjetado) -> tuple[Confianca, str]
```

1. Três fatores, **nesta ordem**:

- a. estimativa do piquete de destino: `(destino.confianca, destino.motivo_confianca)`;
- b. parâmetro de altura: `(destino.parametros.confianca, frase_parametro)`;
- c. peso do lote: `(lote.confianca_peso, lote.motivo_confianca_peso)`.

2. `nível = combinar_confianca(a, b, c)`. A frase é a do **primeiro** fator, na ordem acima, cujo nível é igual a `nível`.

3. `frase_parametro`:

- se `parametros fonte` começa com "produtor:" → "a altura de entrada foi informada por você, não por fonte técnica";
- senão → `f"a altura de entrada do {cultivar_nome} no pastejo {metodo} vem de fonte com confiança {nivel_do_parametro}"`, com `metodo` em `rotacionado/contínuo` e o `nível` em `alta/média/baixa`, com acento.

Critérios de aceite:

- [] Destino alta, parâmetro média, peso média → `("media", "a altura de entrada do Marandu no pastejo rotacionado vem de fonte com confiança média")`.
- [] Destino baixa → a frase do destino.

Fora do escopo: confiança numérica; multiplicar fatores.

J3: O otimizador guloso semanal (planner/otimizador.py)

Por quê: é a entrega central do SeuGado: transformar o estado do pasto numa ordem de manejo que respeita a agronomia e a rotina da fazenda (dias de manejo e mão de obra). Nenhum concorrente modela essa rotina.

Constantes:

```
HORIZONTE_DIAS = 7
FRACAO_ALVO_MARGINAL = 0.9 # HIPOTESE-CALIBRAR (ADR-024)
```

Algoritmo (siga exatamente, porque a fixture depende disso):

Passo 0: preparar.

- `P` = piquetes por id; `L` = lotes por id.
- Um piquete é **planejável** se: `metodo_pastejo == ROTACIONADO`, `faltantes` vazio, `massa_hoje_kg_ms_ha` não nulo e `parametros` não nulo.
- Estado simulado, que você atualiza dia a dia: `massa[p]` (começa em `massa_hoje`), `ocupante[p]` (id do lote ou `None`, a partir de `lote.piquete_atual_id`), `posicao[l]` (o piquete do lote) e `descanso[p]` (começa em `dias_descanso`).
- `capacidade = funcionarios_disponiveis × manejos_por_funcionario_dia`.
- `altura(p) = massa[p] ÷ densidade`.
- `proximo_dia_de_manejo(d)`: primeiro dia depois de `d`, dentro de 7 dias, cujo `weekday()` está nos dias preferenciais (pode cair fora do horizonte do plano; tudo bem, a taxa tem 14 dias).

Passo 1: alertas e pedidos do início (data = `data_base`), por piquete em ordem de nome:

- `faltantes` não vazio:
 - se algum item for de calibração ou altura (`densidade_kg_ha_por_cm`, `rue_max_g_por_mj`, `eficiencia_pastejo`, `altura_entrada_cm`, `altura_saida_cm`, `altura_maxima_cm`, `altura_minima_cm`) → `AGUARDANDO_PARAMETRO`;
 - senão → `ESTIMATIVA_INDISPONIVEL`;
 - texto: "0 {nome} ({cultivar_nome}) ainda não recebe recomendação: {motivo_confianca}.", confiança e motivo do próprio piquete.
- Senão, contínuo com estimativa:
 - `altura_hoje > altura_maxima` → `CONTINUO_ACIMA_MAXIMA`: "0 {nome} (pastejo contínuo) está em {h} cm, acima da altura máxima do {cultivar} ({max} cm). Considere colocar mais animais nele.";
 - `altura_hoje < altura_minima` → `CONTINUO_ABAIXO_MINIMA`: "... abaixo da altura mínima do {cultivar} ({min} cm). Considere tirar animais dele."
- **Pedido de validação** (medir com régua) se: o piquete tem estimativa e confiança baixa, ou "`altura_inicial`" está em `faltantes`. `motivo = motivo_confianca` do piquete.
- Lote sem piquete (`piquete_atual_id` nulo) → `LOTE_SEM_PIQUETE`: "0 lote {nome} não está em nenhum piquete. Informe onde ele está na página Lotes.", com a confiança do peso do lote.

Passo 2: simular a semana, para `i` de 0 a 6 (`d = data_base + i`):

1. Se `d` é dia de manejo:
 - `n = (proximo_dia_de_manejo(d) - d).days`.

- **Candidatos a sair:** cada lote cujo piquete atual é planejável. Simule n dias do piquete com o lote dentro (a partir de `massa[p]`, taxas de índice $i \dots i+n-1$, usando `avancar_massa_um_dia`) $\rightarrow$ `altura_prevista`. Se `altura_prevista <= altura_saida`, o lote é candidato com `urgencia(altura_prevista, parametros)`.
- Ordene os candidatos por urgência **decrecente**; empate $\rightarrow$ nome do lote.
- Para cada candidato, em ordem:
 - Se já houve `capacidade` movimentos hoje $\rightarrow$ alerta `CAPACIDADE_EXCEDIDA`: "Não há mão de obra para mover o lote {lote} {na/no} {dia por extenso} ({dd/mm}): o limite é {cap} movimentação(ões) por dia. Ele continua no {origem}.". Pule para o próximo.
 - **Escolher o destino:** entre os piquetes planejáveis, vazios, diferentes da origem e com `descanso_cumprido(descanso[q], descanso_min_dias)`:
 - **aptos:** `altura(q) >= altura_entrada`. Escolha o menor (`|altura(q) - entrada|`, distância da origem, nome);
 - se não houver apto, **marginais:** `altura(q) >= 0,9 \times entrada`. Escolha o menor (`entrada - altura(q)`, distância da origem, nome);
 - se não houver nenhum $\rightarrow$ alerta `SEM_PIQUETE_APT0`: "Nenhum piquete estará pronto para o lote {lote} {na/no} {dia} ({dd/mm}). Ele continua no {origem}, que deve ficar abaixo de {saida} cm antes de {próximo dia de manejo por extenso} ({dd/mm})." O lote fica onde está.
 - **Registrar a movimentação:**
 - `id = uuid5(plano_id, f"{lote_id}:{d.isoformat()}")`;
 - `altura_destino_cm` e `altura_origem_cm` = altura no início do dia d , com 1 casa;
 - alvos de entrada (destino) e saída (origem);
 - `motivo` = parte 1 + " " + parte 2;
 - `confianca`, `motivo_confianca` = `confianca_movimentacao(destino, lote)`.

Atualize: `ocupante[origem] = None`, `descanso[origem] = 0`, `ocupante[destino] = lote`, `posicao[lote] = destino`.

2. **Avançar um dia:** para cada piquete planejável, `massa[p] = avancar_massa_um_dia(massa[p], taxa[i], consumo do ocupante (0 se vazio), area, eficiencia)`. Depois, `descanso[p] += 1` para todo piquete **vazio**.

Passo 3: dias previstos. Para cada movimentação:

- se o mesmo lote tem uma movimentação **posterior** no plano $\rightarrow$ a diferença em dias;
- senão $\rightarrow$ `floor(dias_ocupacao(massa do destino no início do dia do movimento, altura_saida \times densidade, média das 7 taxas do destino a partir do índice do dia, area, eficiencia, consumo))`. Se der infinito (o capim cresce mais do que o lote come), use 7.

Passo 4: montar o plano.

- `movimentacoes` ordenadas por `(data, lote_nome)`; alertas por `(data, tipo.value, texto)`; `pedidos_validacao` por nome do piquete.
- `piquetes`: um `ResumoPiquete` por piquete, na ordem do estado, com `altura_hoje_cm`, alvos de entrada e saída dos `parametros` (nulos se não houver), nome do lote atual, confiança, motivo

e faltantes.

- `data_geracao = agora, data_inicio = data_base, horizonte_dias = 7.`

Textos (português, prontos para o produtor):

- **Parte 1:** "0 {origem} chegaria a {h_prev} cm antes do próximo dia de manejo ({ddd} {dd/mm}), abaixo da saída de {saida} cm."
- **Parte 2 (apto):** "0 {destino} está em {h} cm (ponto de entrada: {entrada} cm)."
- **Parte 2 (marginal):** "Nenhum piquete estará no ponto de entrada; o {destino} é o mais próximo, com {h} cm (alvo: {entrada} cm)."
- **Números:** 1 casa decimal com vírgula, e sem ,0 no fim (31,5, 34, 15).
- **Dia abreviado:** seg ter qua qui sex sáb dom.
- **Dia por extenso:** segunda terça quarta quinta sexta sábado domingo, com "na" antes (e "no" antes de sábado e domingo).

Crítérios de aceite:

- [] **Fixture:** com o estado de exemplo, a fazenda do exemplo, o `agora` e o `plano_id` da seção 4, `plano_para_dict(gerar_plano(...))` é **igual** a `plano_exemplo.json`.
- [] **Determinismo:** duas chamadas iguais → planos iguais.
- [] **Dia não preferencial:** nenhuma movimentação fora dos dias preferenciais.
- [] **Mão de obra saturada:** com capacidade 1 e 2 lotes urgentes no mesmo dia → 1 movimentação (o mais urgente) + 1 `CAPACIDADE_EXCEDIDA`.
- [] **Resíduo:** nenhum lote é mandado para piquete abaixo de 90% da entrada.
- [] **Sem piquete apto:** gera `SEM_PIQUETE_APT0` e o lote fica.
- [] **Contínuo e faltantes:** nunca recebem nem perdem lote; só geram alerta.
- [] **Confiança baixa em tudo:** ainda produz plano, alertas e pedidos, sem exceção.

Fora do escopo: fusão de lotes (F-022); ajuste de lotação no contínuo (F-009B); CP-SAT ou busca local; lookahead de "esperar dois dias"; regra de 1-3 dias de ocupação (ADR-024); qualquer I/O neste arquivo.

J4: Gravar e ler o plano (persistencia/planos.py)

Por quê: o plano precisa ficar guardado. O site mostra o "plano atual", e o bot precisa saber quais movimentações o produtor está confirmando. Cada movimentação também vira evento `manejo_recomendado`, que é o que permite, depois, comparar o que o sistema mandou com o que o produtor fez.

Como:

```
def salvar_plano(conn, plano: PlanoManejo) -> None
def carregar_plano_atual(conn, fazenda_id: UUID) -> PlanoManejo | None
def carregar_plano(conn, plano_id: UUID) -> PlanoManejo | None
```

1. salvar_plano:

- `INSERT INTO plano (id, fazenda_id, gerado_em, data_inicio, horizonte_dias, payload), COM payload = Json(plano_para_dict(plano)) (from psycopg.types.json import Json);`

- depois, para cada movimentação, `registrar_evento(conn, plano.fazenda_id, TipoEvento.MANEJO_RECOMENDADO, OrigemEvento.SISTEMA, ocorrido_em=plano.data_geracao, payload=..., ator="planner", chave_idempotencia=f"recomendacao:{plano.id}:{mov.id}")`, com o payload `{"entidade_id": mov.id, "lote_id", "piquete_origem_id", "piquete_destino_id", "data_prevista": mov.data, "dias_previstos", "motivo", "confianca", "motivo_confianca"}` (UUIDs e datas como string ISO, confiança como `.value`);
 - **sem commit**: quem chama (o ciclo do Leandro) faz o commit.
2. `carregar_plano_atual`: `SELECT payload FROM plano WHERE fazenda_id = %s ORDER BY gerado_em DESC LIMIT 1` → `plano_de_dict`, OU None.
 3. `carregar_plano`: o mesmo, por `id`.

Critérios de aceite:

- [] Salvar e carregar devolve um plano igual ao original.
- [] Salvar o mesmo plano duas vezes não duplica eventos (o `INSERT` do plano falha na segunda vez; isso é esperado).
- [] Testes com banco pulam se `SEUGADO_TEST_DATABASE_URL` não existir.

Fora do escopo: apagar ou editar plano; histórico paginado.

J5: Testes e PR

1. `tests/planner/test_otimizador.py` com todos os critérios de J3. Monte estados pequenos direto em Python (2 a 4 piquetes) para os casos isolados, além do teste da fixture.
2. `tests/planner/test_confianca.py` e `tests/persistencia/test_planos.py`.
3. `uv run ruff check .`, `uv run mypy`, `uv run --env-file .env pytest`.
4. PR `feat/joao-otimizador` → `main`, com a saída do plano de exemplo no texto do PR.

6. Quem usa o seu código

- **Leandro**, no ciclo semanal:

```
plano = gerar_plano(estado, fazenda, datetime.now(UTC), uuid4())
salvar_plano(conn, plano)
conn.commit()
```

E também `carregar_plano_atual` para a página do plano.

- **Leo** lê as movimentações pelo `id` para os botões “Fiz / Não fiz / Fiz diferente” e usa `carregar_plano_atual` e `carregar_plano`.
- **Ezequiel** pinta o mapa com `plano.piquetes` e desenha as setas com `plano.movimentacoes`.

Por isso a saída precisa bater **exatamente** com o contrato: todo mundo desenvolve hoje em cima da fixture.

7. Usando IA no seu fluxo

Dê à IA este documento, `src/seugado/contratos.py` e as duas fixtures, e peça uma tarefa por vez. Exemplo:

“Leia docs/equipe/JOAO.md (seções 4 e J3), `src/seugado/contratos.py` e `tests/fixtures/*.json`. Implemente `src/seugado/planner/otimizador.py` seguindo o algoritmo passo a passo. O teste principal é reproduzir `plano_exemplo.json` exatamente. Não altere nenhum outro arquivo e não adicione dependências.”

Se o seu resultado divergir da fixture, compare movimento a movimento. As diferenças mais comuns: esquecer de contar descanso só para piquete **vazio**, usar a altura do fim do dia em vez do início, e formatar `34,0` em vez de `34`.

8. Glossário de conferência (números da fixture)

- **Segunda 28/09:** o lote Recria (150 novilhos, 1.113,75 kg MS/dia) está no Piquete 2 (22 cm, 3,5 ha). Até quinta, cairia para 11,6 cm, abaixo da saída de 15 cm, então sai. O Piquete 1 (31,5 cm) e o 6 (34 cm) estão aptos; o 1 está mais perto do alvo de 30 cm, e o Recria, mais urgente, vai para ele. As Vacas com bezerro vão para o 6.
- **Quinta 01/10:** os dois lotes cairiam abaixo de 15 cm até a segunda seguinte. Nenhum piquete está a 30 cm, mas o 3 está a 28,6 cm (≥ 27 , que é 90% de 30) e fica com o Recria, que é o mais urgente. As Vacas ficam, com alerta.